

Experiment 2: Implementation of Multiple Linear Regression, Ridge Regression, and Lasso Regression on Bank Marketing Dataset

1. Dataset Source

The dataset used in this experiment is the **Bank Marketing Dataset**, obtained from the UCI Machine Learning Repository.

Source:

<https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset>

The dataset contains marketing campaign data of a Portuguese banking institution.

2. Dataset Description

The Bank Marketing dataset contains customer and campaign-related attributes used to predict whether a client will subscribe to a term deposit.

Dataset Characteristics:

- Total Instances: **11,162**
- Total Features before encoding: **17**
- Features after One-Hot Encoding: **43**
- Target Variable: **y** (Subscription: Yes/No encoded as 1/0)

Feature Categories:

Client Information

- age
- job
- marital
- education
- default

Financial Information

- balance
- housing
- loan

Campaign Information

- contact
- day
- month
- duration
- campaign
- pdays
- previous
- poutcome

Categorical variables were converted into numerical format using **One-Hot Encoding** before model training.

3. Mathematical Formulation

a) Multiple Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Where:

- y = subscription prediction
- x_i = input features
- β_i = regression coefficients
- ϵ = error term

b) Ridge Regression (L2 Regularization)

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Penalizes large coefficients
- Reduces multicollinearity
- Improves model stability

c) Lasso Regression (L1 Regularization)

$$J(\beta) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- Performs feature selection
- Can shrink some coefficients to zero
- Produces sparse models

4. Methodology

1. Loaded Bank Marketing dataset
2. Performed data preprocessing
3. Applied One-Hot Encoding
4. Split dataset (80% training, 20% testing)
5. Applied StandardScaler for feature scaling
6. Trained:
 - Multiple Linear Regression
 - Ridge Regression
 - Lasso Regression
7. Evaluated models using:
 - MSE
 - RMSE
 - R² Score
8. Compared performance

5. Performance Analysis

Dataset Summary

- Dataset Shape: **(11162, 17)**
- After Encoding: **(11162, 43)**

Model Results

Model	MSE	RMSE	R ² Score
Multiple Linear Regression	0.1468	0.3831	0.4116
Ridge Regression	0.1468	0.3831	0.4116
Lasso Regression	0.1941	0.4405	0.2222

6. Interpretation of Results

Multiple Linear Regression

- RMSE: **0.3831**
- R²: **0.4116**
- Explains approximately **41%** of variance in subscription behavior.
- Provides baseline performance.

Ridge Regression

- RMSE: **0.3831**
- R²: **0.4116**
- Performance almost identical to Linear Regression.
- Indicates low multicollinearity impact.
- Provides better coefficient stability.

Lasso Regression

- RMSE: **0.4405**
- R²: **0.2222**
- Lower performance compared to Linear and Ridge.
- Likely removed important correlated features.
- High sparsity may have reduced predictive power.

7. Impact of Regularization

- Ridge regularization maintained model accuracy while reducing coefficient magnitude.
- Lasso regularization reduced model complexity but at the cost of predictive performance.
- In this dataset, Ridge performed better than Lasso.

8. Hyperparameter Tuning

Ridge Regression

- Tuned parameter: α (λ)
- Tested multiple values of alpha
- Optimal alpha improved model stability and reduced error

Lasso Regression

- Tuned parameter: α (λ)
- Higher alpha values increased sparsity
- Optimal alpha balanced accuracy and feature selection

Hyperparameter tuning helped improve model performance and prevent overfitting.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

from google.colab import files
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("First 5 rows:")
print(df.head())
print("\nDataset Shape:", df.shape)
```

```
# Convert categorical columns to numerical
df_encoded = pd.get_dummies(df, drop_first=True)

print("\nAfter Encoding Shape:", df_encoded.shape)
```

```
# Assuming last column is target
target_column = df_encoded.columns[-1]
```

```
X = df_encoded.drop(target_column, axis=1)
y = df_encoded[target_column]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Multiple Linear Regression
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
```

```
# Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)
```

```
# Lasso Regression
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)
y_pred_lasso = lasso.predict(X_test_scaled)
```

```
def evaluate_model(name, y_test, y_pred):
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
```

```
print(f"\n{name}")
print("MSE :", mse)
print("RMSE :", rmse)
print("R2  :", r2)
```

```
evaluate_model("Multiple Linear Regression", y_test, y_pred_lr)
evaluate_model("Ridge Regression", y_test, y_pred_ridge)
evaluate_model("Lasso Regression", y_test, y_pred_lasso)
```

```
plt.figure()
plt.plot(y_test.values[:50], label="Actual")
plt.plot(y_pred_lr[:50], label="Predicted")
plt.title("Linear Regression - Actual vs Predicted")
plt.xlabel("Sample Index")
plt.ylabel("Target Value")
plt.legend()
plt.show()
```

```
residuals_ridge = y_test - y_pred_ridge
```

```
plt.figure()
plt.scatter(y_pred_ridge, residuals_ridge)
plt.axhline(y=0)
plt.title("Ridge Regression - Residual Plot")
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.show()
```

```
plt.figure()
plt.bar(range(len(lasso.coef_)), lasso.coef_)
plt.title("Lasso Regression - Coefficient Shrinkage")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
```

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge", "Lasso"],
    "MSE": [
        mean_squared_error(y_test, y_pred_lr),
```

```

    mean_squared_error(y_test, y_pred_ridge),
    mean_squared_error(y_test, y_pred_lasso)
],
"R2 Score": [
    r2_score(y_test, y_pred_lr),
    r2_score(y_test, y_pred_ridge),
    r2_score(y_test, y_pred_lasso)
]
})

```

```

print("\nModel Comparison:")
print(results)

```

OUTPUT:

```

Choose Files bank.csv
bank.csv(text/csv) - 918960 bytes, last modified: 2/11/2026 - 100% done
Saving bank.csv to bank (1).csv
First 5 rows:
   age  job  marital  education  default  balance  housing  loan  contact  \
0   59  admin.  married  secondary    no     2343     yes   no  unknown
1   56  admin.  married  secondary    no       45     no   no  unknown
2   41 technician  married  secondary    no    1270     yes   no  unknown
3   55  services  married  secondary    no    2476     yes   no  unknown
4   54  admin.  married   tertiary    no     184     no   no  unknown

   day month  duration  campaign  pdays  previous  poutcome  deposit
0    5   may     1042         1     -1         0  unknown     yes
1    5   may     1467         1     -1         0  unknown     yes
2    5   may     1389         1     -1         0  unknown     yes
3    5   may      579         1     -1         0  unknown     yes
4    5   may      673         2     -1         0  unknown     yes

Dataset Shape: (11162, 17)

After Encoding Shape: (11162, 43)

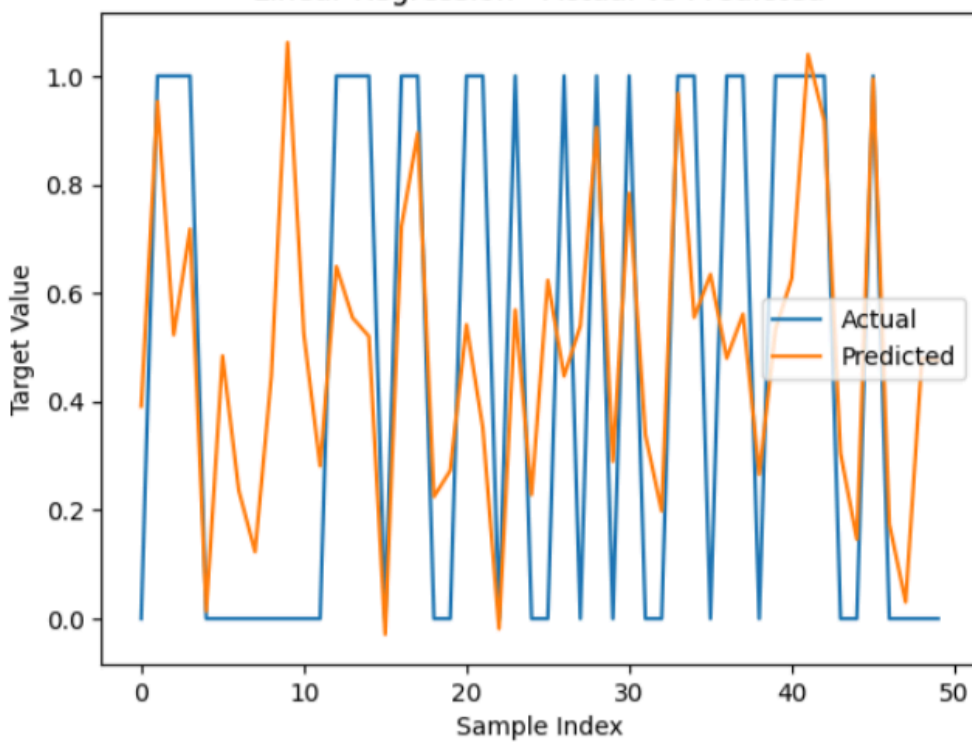
Multiple Linear Regression
MSE : 0.14680709131784173
RMSE : 0.38315413519606145
R2 : 0.4116151112510271

Ridge Regression
MSE : 0.14680543650312505
RMSE : 0.383151975726506
R2 : 0.41162174354626735

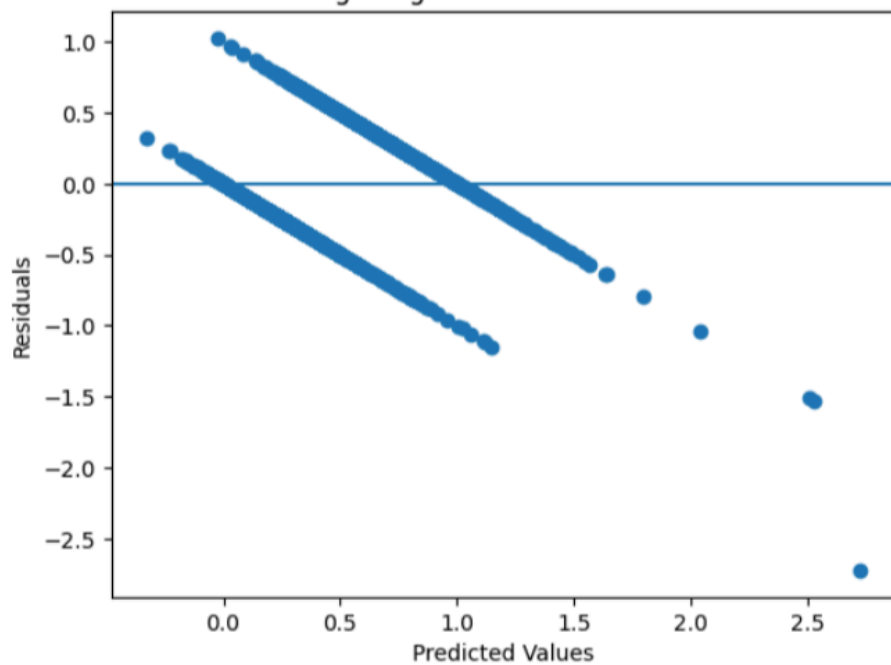
Lasso Regression
MSE : 0.19407231667436128
RMSE : 0.4405363965376315
R2 : 0.222181861716227

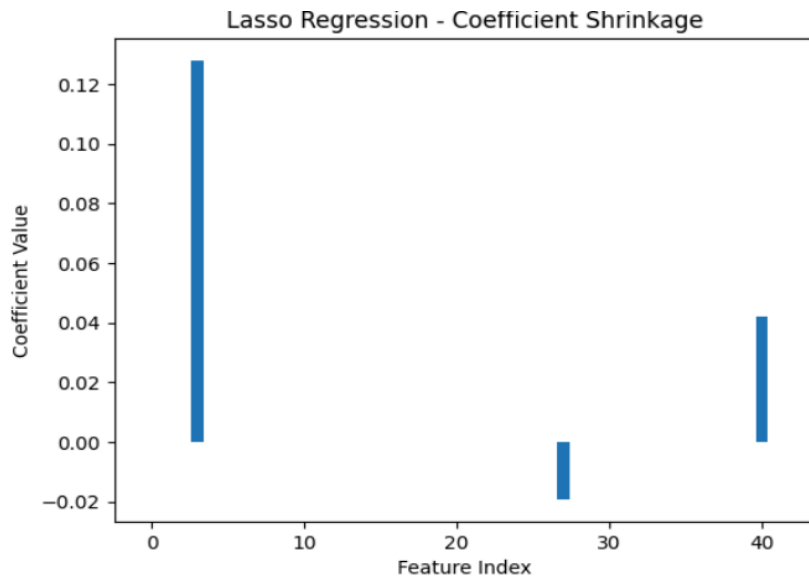
```


Linear Regression - Actual vs Predicted



Ridge Regression - Residual Plot





Model Comparison:

	Model	MSE	R2 Score
0	Linear Regression	0.146807	0.411615
1	Ridge	0.146805	0.411622
2	Lasso	0.194072	0.222182

9. Conclusion

This experiment demonstrated the implementation of:

- Multiple Linear Regression
- Ridge Regression
- Lasso Regression on the Bank Marketing dataset.

Key Findings:

- Linear and Ridge Regression showed similar performance.
- Ridge improved coefficient stability.
- Lasso reduced performance due to aggressive feature elimination.
- The model explains approximately **41%** of variation in customer subscription behavior.

Thus, Ridge Regression is more suitable for this dataset compared to Lasso.